



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA DEPARTAMENTO DE
SISTEMAS Y COMPUTACIÓN

SEMESTRE 9

Enero - Junio 2026

Ingeniería en Sistemas Computacionales

Patrones de diseño

Actividad de evidencia 2_U2 Singleton

Diaz Morales Katherine Giselle 22210302

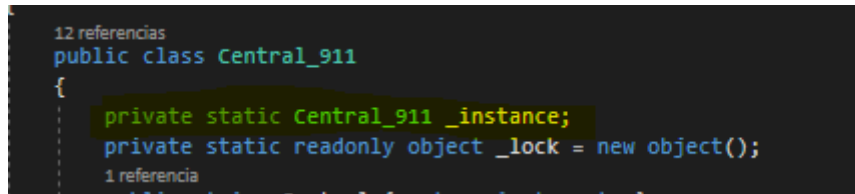
Maribel Guerrero Luis

Tijuana Baja California, a 09 de marzo del 2026

Atributo estático de la instancia

```
private static Central_911 _instance;
```

Esta clase pertenece a la clase y no a los objetos, además almacena a la única instancia que existe en la clase y al ser static es compartida por todas las llamadas

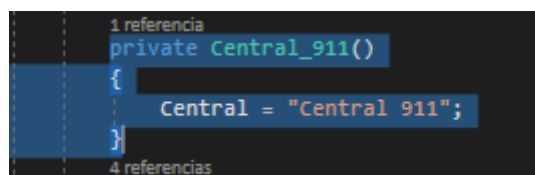


```
12 referencias
public class Central_911
{
    private static Central_911 _instance;
    private static readonly object _lock = new object();
    1 referencia
    public static Central_911 GetInstance()
    {
        lock (_lock)
        {
            if (_instance == null)
            {
                _instance = new Central_911();
            }
            return _instance;
        }
    }
}
```

Constructor

```
private Central_911() {Central = "Central 911"; }
```

Solo puede ser llamado desde dentro de la propia clase y nadie pueda usar el new Central_911() para crear instancias

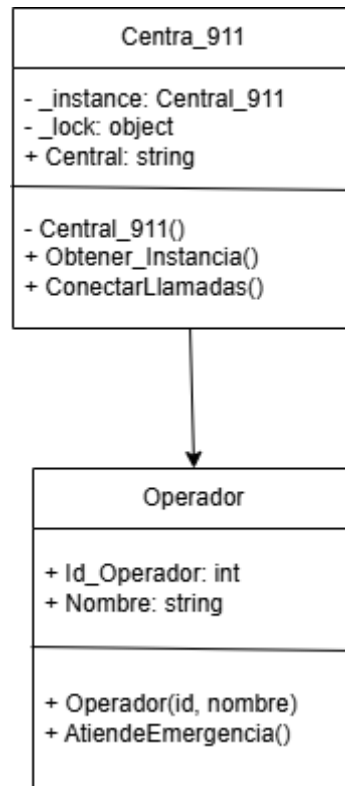


```
1 referencia
private Central_911()
{
    Central = "Central 911";
}
4 referencias
```

¿Por qué no se pueden crear múltiples objetos?

Porque por ejemplo, private impide el uso de new fuera de su clase, static mantiene una sola referencia a si misma y la creación de Obtener_Instance() sólo se crea una vez y siempre devuelve la misma

Diagrama UML



Código

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _2_U2_Singleton
{
    public class Central_911
    {
        private static Central_911 _instance;
        private static readonly object _lock = new object();
        public string Central { get; private set; }
        private Central_911()
    }
}
```

```

{
    Central = "Central 911";
}
public static Central_911 Obtener_Instance()
{
    if(_instance==null)
    {
        lock(_lock)
        {
            if(_instance==null)
            {
                _instance = new Central_911();
            }
        }
    }
    return _instance;
}

public void ConectarLlamadas(Operador operador, string tipoEmergencia)
{
    Console.WriteLine("\nLlamada conectada con el operador " + operador.Nombre);
    operador.AtiendeEmergencia(tipoEmergencia);
}
public class Operador
{
    public int Id_Operador { get; set; }
    public string Nombre { get; set; }
    public Operador(int id, string nombre)
    {
        Id_Operador = id;
        Nombre = nombre;
    }
    public void AtiendeEmergencia(string tipoEmergencia)
    {
        Console.WriteLine($"Operador {Nombre} atendiendo emergencia de tipo: {tipoEmergencia}");
        switch (tipoEmergencia)
        {
            case "Intento de suicidio":
                Console.WriteLine("Enviando unidades de apoyo y rescate");
                break;
            case "Incendio":
                Console.WriteLine("Enviando bomberos");
                break;
            case "Accidente":
                Console.WriteLine("Enviando paramedicos y oficiales");
                break;
            case "Violeta":
                Console.WriteLine("Enviando una patrulla");
                break;
            case "Robo":
                Console.WriteLine("Enviando patrulla");

```

```

        break;
    case "Medica":
        Console.WriteLine("Enviando ambulancia y paramedicos");
        break;
    default:
        Console.WriteLine("Tipo de emergencia no reconocido");
        break;
    }
}
internal class Program
{
    static void Main(string[] args)
    {
        Central_911 Llamada1 = Central_911.Obtener_Instancia();
        Central_911 Llamada2 = Central_911.Obtener_Instancia();
        Central_911 Llamada3 = Central_911.Obtener_Instancia();
        Central_911 Llamada4 = Central_911.Obtener_Instancia();

        Operador op1 = new Operador(1, "Laura");
        Operador op2 = new Operador(2, "Carlos");
        Operador op3 = new Operador(3, "Juan");
        Operador op4 = new Operador(4, "Maria");

        Llamada1.ConectarLlamadas(op1, "Incendio");
        Llamada2.ConectarLlamadas(op2, "Violeta");
        Llamada1.ConectarLlamadas(op1, "Accidente");
        Llamada2.ConectarLlamadas(op2, "Intento de suicidio");
        Llamada3.ConectarLlamadas(op3, "Robo");
        Llamada4.ConectarLlamadas(op4, "Medica");

        Console.WriteLine("\nReferenceEquals: " + ReferenceEquals(Llamada1, Llamada2));

        Console.ReadKey();
    }
}
}
}
}
}
}
}
}

```

```
Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Incendio
Enviando bomberos
$
Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Violeta
Enviando una patrulla
$
Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Accidente
Enviando paramedicos y oficiales
$
Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Intento de suicidio
Enviando unidades de apoyo y rescate
$
Llamada conectada con el operador Juan
Operador Juan atendiendo emergencia de tipo: Robo
Enviando patrulla
$
Llamada conectada con el operador Maria
Operador Maria atendiendo emergencia de tipo: Medica
Enviando ambulancia y paramedicos
$
ReferenceEquals: True
$
```

Conclusión

Con esta actividad refuerce el funcionamiento del patrón Singleton y me di cuenta de que al crear varias llamadas siempre se termina usando la misma instancia gracias al `private` y `static`, además singleton ayudó a controlar el acceso a un recurso compartido y al tener una sola central de emergencias todo se centraliza evitando conflictos.